

Unit IV Generative Models and Optimization

Generative Models – Introduction to Generative Models – GANs – VAEs – Variational Autoencoders (VAEs) – Generative Adversarial Networks (GANs) – Basic GANs – Conditional GANs – Advanced Variants. Optimization and Regularization Techniques – Optimization Algorithms – Adam – RMSprop – Momentum – Regularization Methods – Dropout – Batch Normalization – Weight Decay – Hyperparameter Tuning.

Generative Models:

Generative models are a class of machine learning models designed to create new data instances that resemble the training data. They learn the underlying patterns and distributions of data, enabling them to generate realistic images, text, music, and more.

Types of Generative Models

1. **Generative Adversarial Networks (GANs)** – Uses a generator and discriminator in a competitive framework to create realistic outputs.
2. **Variational Autoencoders (VAEs)** – Encodes data into a latent space and reconstructs it to generate new samples.
3. **Diffusion Models** – Gradually refine noisy data to generate high-quality images.
4. **Autoregressive Models** – Predict the next data point based on previous ones (e.g., GPT for text generation).

Applications of Generative Models

- **Image Generation** – AI-generated art, deepfake technology.
- **Text Generation** – Chatbots, automated content creation.
- **Music Composition** – AI-generated melodies and soundtracks.
- **Drug Discovery** – Designing new molecular structures.
- **Data Augmentation** – Enhancing datasets for better model training.

Generative models are shaping the future of AI by enabling machines to create content that mimics human creativity

Introduction to Generative Models

Generative models are a class of machine learning models designed to create new data instances that resemble the training data. They learn the underlying patterns and distributions of data, enabling them to generate realistic images, text, music, and more.

How Generative Models Work

Generative models aim to understand the statistical distribution of a dataset and generate new samples that follow the same patterns. They can be trained using techniques like:

- **Maximum Likelihood Estimation (MLE)** – Learning the probability distribution of data.
- **Adversarial Learning** – Competing networks refine generated outputs.
- **Latent Space Representation** – Mapping data into a compressed form for generation.

Types of Generative Models

1. **Generative Adversarial Networks (GANs)** – Uses a generator and discriminator in a competitive framework to create realistic outputs.
2. **Variational Autoencoders (VAEs)** – Encodes data into a latent space and reconstructs it to generate new samples.
3. **Diffusion Models** – Gradually refine noisy data to generate high-quality images.
4. **Autoregressive Models** – Predict the next data point based on previous ones (e.g., GPT for text generation).

Applications of Generative Models

- **Image Generation** – AI-generated art, deepfake technology.
- **Text Generation** – Chatbots, automated content creation.
- **Music Composition** – AI-generated melodies and soundtracks.
- **Drug Discovery** – Designing new molecular structures.
- **Data Augmentation** – Enhancing datasets for better model training.

Generative models are shaping the future of AI by enabling machines to create content that mimics human creativity

GAN:

Generative Adversarial Networks (GANs) are a class of machine learning models designed to generate realistic data by learning from existing examples. Introduced by **Ian Goodfellow** in 2014, GANs consist of two competing neural networks: a **generator** and a **discriminator**.

How GANs Work

1. **Generator** – Creates synthetic data (e.g., images, text) from random noise.
2. **Discriminator** – Evaluates whether the generated data is real or fake.
3. **Adversarial Training** – The generator tries to fool the discriminator, while the discriminator improves at distinguishing real from fake data.

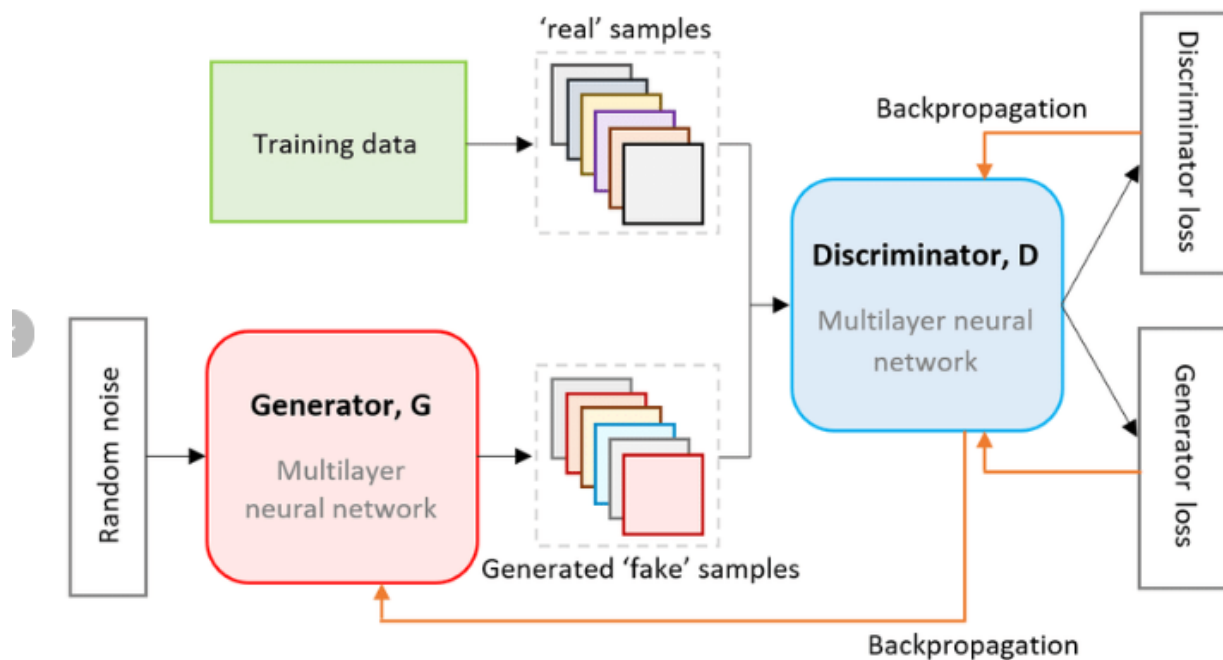
Key Features

- **Unsupervised Learning** – GANs learn patterns without labeled data.
- **High-Quality Generation** – Used for realistic image synthesis.
- **Adversarial Process** – Continuous improvement through competition.

Applications of GANs

- **Image Generation** – AI-generated art, deepfake technology.
- **Text Generation** – Chatbots, automated content creation.
- **Music Composition** – AI-generated melodies and soundtracks.
- **Data Augmentation** – Enhancing datasets for better model training.

Architecture Diagram of GAN:



VariationalAutoencoders :

VariationalAutoencoders (VAEs) are a type of generative model that extends traditional autoencoders by learning a **probabilistic representation** of data. Unlike standard autoencoders, which compress and reconstruct data, VAEs generate new, realistic samples by modeling the underlying distribution of the dataset.

How VAEs Work

1. **Encoder** – Maps input data to a latent space, producing a mean and variance instead of a fixed representation.

2. **Latent Space Sampling** – Introduces randomness by sampling from the learned distribution, enabling diverse outputs.
3. **Decoder** – Reconstructs the original data from the sampled latent vector, generating new variations.

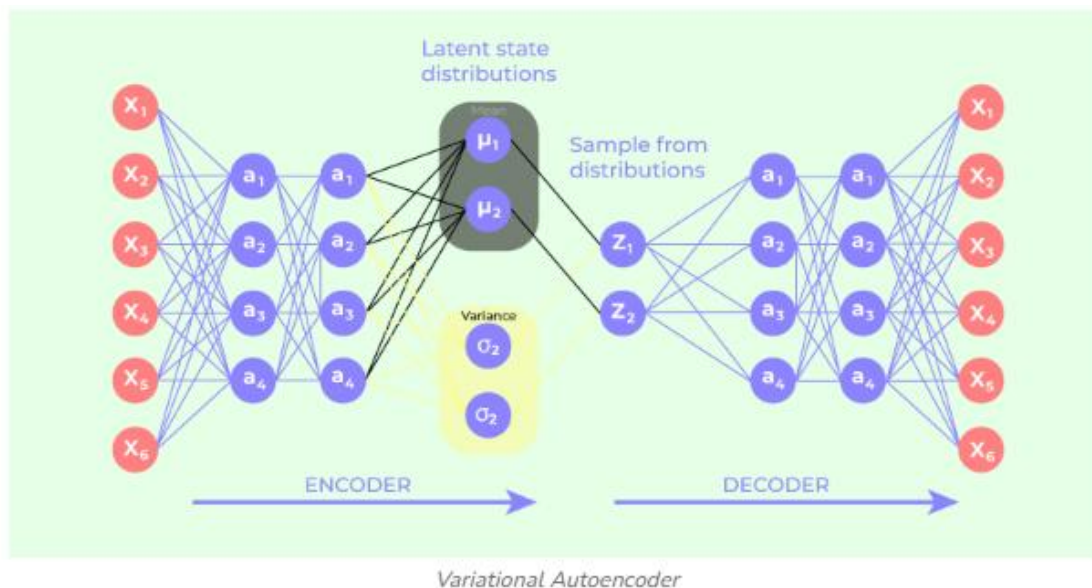
Key Features

- **Probabilistic Representation** – Instead of encoding data as a single point, VAEs learn a distribution.
- **Reparameterization Trick** – Ensures smooth training by making sampling differentiable.
- **Generative Capability** – Can create new, realistic data samples.

Applications of VAEs

- **Image Generation** – AI-generated art, deepfake technology.
- **Data Augmentation** – Enhancing datasets for better model training.
- **Anomaly Detection** – Identifying fraudulent transactions or faulty sensor readings.
- **Drug Discovery** – Designing new molecular structures.

Architecture diagram of Variational Autoencoders



Basic Generative Adversarial Networks (GANs)

Generative Adversarial Networks (GANs) are a type of deep learning model designed to generate realistic data by learning from existing examples. Introduced by **Ian Goodfellow** in 2014, GANs consist of two competing neural networks: a **generator** and a **discriminator**.

How GANs Work

1. **Generator** – Creates synthetic data (e.g., images, text) from random noise.
2. **Discriminator** – Evaluates whether the generated data is real or fake.
3. **Adversarial Training** – The generator tries to fool the discriminator, while the discriminator improves at distinguishing real from fake data.

Key Features

- **Unsupervised Learning** – GANs learn patterns without labeled data.
- **High-Quality Generation** – Used for realistic image synthesis.
- **Adversarial Process** – Continuous improvement through competition.

Applications of GANs

- **Image Generation** – AI-generated art, deepfake technology.
- **Text Generation** – Chatbots, automated content creation.
- **Music Composition** – AI-generated melodies and soundtracks.
- **Data Augmentation** – Enhancing datasets for better model training.

Conditional Gan:

Conditional GANs (cGANs) are an extension of **Generative Adversarial Networks (GANs)** that allow the generation of data based on specific conditions, such as class labels or descriptions. Unlike standard GANs, which produce random outputs, cGANs control the generation process by incorporating additional information.

How cGANs Work

1. **Generator** – Takes both **random noise** and **conditioning information** (e.g., labels) to create realistic data.
2. **Discriminator** – Evaluates whether the generated data is real or fake while considering the conditioning information.
3. **Adversarial Training** – The generator improves by fooling the discriminator, while the discriminator learns to distinguish real from fake data.

Key Features

- **Controlled Generation** – Allows targeted output generation (e.g., generating images of specific objects).
- **Improved Training Stability** – Conditioning helps guide the learning process.
- **Versatile Applications** – Used in image synthesis, text generation, and style transfer.

Applications of cGANs

- **Image Generation** – Creating images based on specific labels (e.g., generating only "Mercedes" cars).
- **Text-to-Image Synthesis** – Generating images from textual descriptions.
- **Data Augmentation** – Enhancing datasets for better model training.
- **Style Transfer** – Modifying images based on given attributes.

For a deeper dive into cGANs, check out this resource. Would you like to explore how cGANs compare to standard GANs?

conditionalGan network:

Conditional Generative Adversarial Networks (cGANs)

Conditional GANs (cGANs) are an extension of **Generative Adversarial Networks (GANs)** that allow the generation of data based on specific conditions, such as class labels or descriptions. Unlike standard GANs, which produce random outputs, cGANs control the generation process by incorporating additional information.

How cGANs Work

1. **Generator** – Takes both **random noise** and **conditioning information** (e.g., labels) to create realistic data.
2. **Discriminator** – Evaluates whether the generated data is real or fake while considering the conditioning information.
3. **Adversarial Training** – The generator improves by fooling the discriminator, while the discriminator learns to distinguish real from fake data.

Key Features

- **Controlled Generation** – Allows targeted output generation (e.g., generating images of specific objects).
- **Improved Training Stability** – Conditioning helps guide the learning process.
- **Versatile Applications** – Used in image synthesis, text generation, and style transfer.

Applications of cGANs

- **Image Generation** – Creating images based on specific labels (e.g., generating only "Mercedes" cars).
- **Text-to-Image Synthesis** – Generating images from textual descriptions.
- **Data Augmentation** – Enhancing datasets for better model training.
- **Style Transfer** – Modifying images based on given attributes.

Advanced Variants of GANs

Generative Adversarial Networks (**GANs**) have evolved significantly since their introduction, leading to various advanced architectures that improve stability, efficiency, and quality of generated data. Here are some notable variants:

1. Progressive GANs

- Generates images **gradually**, starting with low resolution and refining details in later layers.
- Improves training stability and enables **high-resolution image generation**.

2. Conditional GANs (cGANs)

- Allows controlled generation by incorporating **labels or attributes** into the model.
- Used for **text-to-image synthesis** and **style transfer**.

3. CycleGAN

- Performs **image-to-image translation** without requiring paired datasets.
- Used for **style transformations**, such as converting horse images into zebra images.

4. StyleGAN

- Introduces **style-based generation**, allowing fine-grained control over features like facial expressions and textures.
- Used in **AI-generated portraits and fashion design**.

5. Super-Resolution GANs

- Enhances image resolution by adding realistic details to low-quality images.
- Used in **medical imaging and satellite image enhancement**.

GANs continue to push the boundaries of AI creativity, enabling applications in **art, gaming, healthcare, and beyond**

Optimization and Regularization Techniques in GANs

Generative Adversarial Networks (**GANs**) are powerful but notoriously difficult to train due to issues like **mode collapse, vanishing gradients, and instability**. To improve training, researchers have developed various **optimization and regularization techniques**.

Optimization Techniques

1. **Adam Optimizer** – Works better than standard SGD by adapting learning rates.
2. **Two Time-Scale Update Rule (TTUR)** – Uses different learning rates for the generator and discriminator.
3. **Gradient Penalty (WGAN-GP)** – Helps stabilize training by enforcing Lipschitz continuity.
4. **Spectral Normalization** – Constrains the discriminator's weights to prevent overfitting.
5. **Instance Noise** – Adds noise to inputs to prevent the discriminator from becoming too confident.

Regularization Techniques

1. **Dropout** – Reduces overfitting by randomly disabling neurons during training.
2. **Batch Normalization** – Helps stabilize training and improve convergence.
3. **Weight Clipping** – Limits weight values to prevent extreme updates.
4. **Feature Matching** – Ensures the generator produces diverse outputs.
5. **Unrolled GAN** – Allows the generator to anticipate discriminator updates.

Optimization algorithms :

Both **Adam** and **RMSProp** are advanced optimization algorithms used in deep learning to improve training efficiency and convergence speed.

1. RMSProp (Root Mean Square Propagation)

- Adapts the learning rate for each parameter individually based on the moving average of squared gradients.
- Helps prevent the learning rate from becoming too small, addressing issues in **AdaGrad**.
- Commonly used in **recurrent neural networks (RNNs)** and deep learning models with non-stationary objectives.

2. Adam (Adaptive Moment Estimation)

- Combines **RMSProp** and **momentum-based optimization** to improve gradient updates.
- Uses **first-moment (mean) and second-moment (variance) estimates** to adjust learning rates dynamically.
- Works well for **complex neural networks** and large datasets.

Key differences:

Feature	RMSProp	Adam
Learning Rate Adaptation	Yes	Yes
Momentum	No	Yes
Stability	Good for RNNs	Works well for most models
Computational Cost	Lower	Slightly higher

Rms Prop algorithm :

RMSProp (Root Mean Square Propagation) is an adaptive learning rate optimization algorithm designed to improve the performance and speed of training deep learning models. It was introduced by **Geoff Hinton** in his Coursera lecture on **Neural Networks for Machine Learning**.

How RMSProp Works

1. **Gradient Computation** – Calculates the gradient of the loss function with respect to model parameters.
2. **Moving Average of Squared Gradients** – Maintains an exponentially decaying average of past squared gradients.
3. **Adaptive Learning Rate** – Adjusts the learning rate for each parameter individually, preventing drastic changes.
4. **Parameter Update** – Uses the adjusted learning rate to update model weights.

Mathematical Formulation

Given a gradient g_t at time step t :

- Compute the moving average of squared gradients:

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1 - \gamma)g_t^2$$

where γ is the decay rate.

- Update the parameter θ using the adjusted learning rate:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

where η is the learning rate and ϵ is a small constant for numerical stability.

Advantages of RMSProp

- **Handles Non-Stationary Objectives** – Works well with dynamic learning environments.
- **Prevents Learning Rate Decay** – Unlike **AdaGrad**, RMSProp avoids overly small learning rates.
- **Efficient for Deep Learning** – Commonly used in **RNNs** and **CNNs**.

Applications

- **Training Deep Neural Networks** – Used in NLP, image recognition, and reinforcement learning.
- **Optimizing Complex Models** – Helps stabilize training in deep architectures.

Adam Optimisers :

Adam (Adaptive Moment Estimation) is a widely used optimization algorithm in deep learning. It combines the benefits of **Momentum** and **RMSProp**, making it effective for training complex neural networks.

How Adam Works

1. **Gradient Computation** – Calculates the gradient of the loss function.
2. **First Moment Estimate (Momentum)** – Tracks the moving average of past gradients.
3. **Second Moment Estimate (Variance)** – Maintains an exponentially weighted average of squared gradients.
4. **Bias Correction** – Adjusts estimates to prevent bias in early training steps.
5. **Parameter Update** – Uses adaptive learning rates for each parameter.

Final Weight Update :

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{\theta}_t + \epsilon}} \hat{m}_t$$

Advantages of Adam

- **Adaptive Learning Rates** – Adjusts learning rates dynamically.
- **Handles Sparse Data** – Works well with noisy gradients.
- **Efficient for Large Models** – Reduces computational overhead.

Applications

- **Deep Learning** – Used in CNNs, RNNs, and Transformers.
- **Natural Language Processing (NLP)** – Improves text generation and translation.
- **Computer Vision** – Enhances image classification and object detection.

RMSProp Optimizer in Deep Learning

Last Updated : 23 May, 2025

•

RMSProp (Root Mean Square Propagation) is an adaptive learning rate optimization algorithm designed to improve the performance and speed of training deep learning models.

- It is a variant of the **gradient descent** algorithm which adapts the learning rate for each parameter individually by considering the magnitude of recent gradients for those parameters.
- This adaptive nature helps in dealing with the challenges of non-stationary objectives and sparse gradients commonly encountered in deep learning tasks.

Need of RMSProp Optimizer

RMSProp was developed to address the limitations of previous optimization methods such as **SGD (Stochastic Gradient Descent)** and **AdaGrad** as SGD uses a constant learning rate which can be inefficient and AdaGrad reduces the learning rate too aggressively.

RMSProp balances by adapting the learning rates based on a moving average of squared gradients. This approach helps in maintaining a balance between efficient convergence and stability during the training process making RMSProp a widely used optimization algorithm in modern deep learning.

How RMSProp Works?

RMSProp keeps a moving average of the squared gradients to normalize the gradient updates. By doing so it prevents the learning rate from becoming too small which was a **drawback in AdaGrad** and ensures that the updates are appropriately scaled for each parameter. This

mechanism allows RMSProp to perform well even in the presence of non-stationary objectives, making it suitable for training deep learning models.

The mathematical formulation is as follows:

1. Compute the gradient g_t at time step t :

$$g_t = \nabla \theta$$

2. Update the moving average of squared gradients:

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1-\gamma)g_t^2$$

where γ is the decay rate.

3. Update the parameter θ using the adjusted learning rate:

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{E[g^2]_t + \epsilon}}$$

where η is the learning rate and ϵ is a small constant added for numerical stability.

Parameters Used in RMSProp

- **Learning Rate (η):** Controls the step size during the parameter updates. RMSProp typically uses a default learning rate of 0.001, but it can be adjusted based on the specific problem.
- **Decay Rate (γ):** Determines how quickly the moving average of squared gradients decays. A common default value is 0.9, which balances the contribution of recent and past gradients.
- **Epsilon (ϵ):** A small constant added to the denominator to prevent division by zero and ensure numerical stability. A typical value for ϵ is $1e-8$.

By carefully adjusting these parameters, RMSProp effectively adapts the learning rates during training, leading to faster and more reliable convergence in deep learning models.

Implementing RMSprop in Python using TensorFlow or Keras

We will use the following code line for initializing the RMSProp optimizer with hyperparameters:

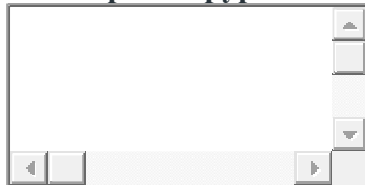
`tf.keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9)`

- **learning_rate=0.001:** Sets the step size for weight updates. Smaller learning rates result in smaller updates, helping to fine-tune weights and prevent overshooting the minimum loss.
- **rho=0.9:** The discounting factor for the history of gradients, controlling the influence of past gradients on the current gradient computation.

1. Importing Libraries

We are importing libraries to implement RMSProp optimizer, handle datasets, build the model and plot results.

- **tensorflow.keras** for deep learning components.
- **matplotlib.pyplot** for visualization.

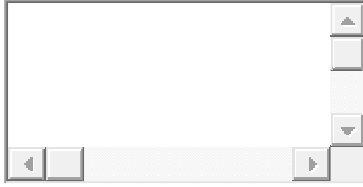


```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
```

2. Loading and Preprocessing Dataset

We load the MNIST dataset, normalize pixel values to [0,1] and [one-hot encode labels](#).

- `mnist.load_data()` loads images and labels.
- **Normalization** improves training stability.
- `to_categorical()` converts labels to one-hot vectors.



```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
x_train = x_train.astype('float32') / 255.0
```

```
x_test = x_test.astype('float32') / 255.0
```

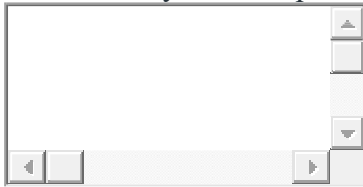
```
y_train = to_categorical(y_train, 10)
```

```
y_test = to_categorical(y_test, 10)
```

3. Building the Model

We define a neural network using Sequential with input flattening and dense layers.

- **Flatten** converts 2D images to 1D vectors.
- Dense layers learn patterns with [ReLU](#) and [softmax](#) activations.

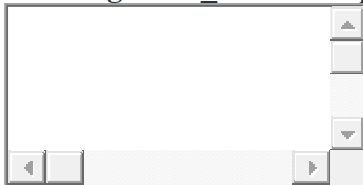


```
model = Sequential([  
    Flatten(input_shape=(28, 28)),  
    Dense(128, activation='relu'),  
    Dense(64, activation='relu'),  
    Dense(10, activation='softmax')  
])
```

4. Compiling the Model

We compile the model using the RMSprop optimizer for adaptive learning rates, categorical cross-entropy loss for multi-class classification and track accuracy metric.

- **RMSprop** adjusts learning rates based on recent gradients (parameter rho controls decay rate).
- **categorical_crossentropy** suits one-hot encoded labels.

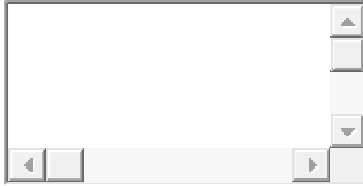


```
model.compile(optimizer=tf.keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9),  
              loss='categorical_crossentropy',
```

```
metrics=['accuracy'])
```

5. Training the Model

We train the model over 10 epochs with batch size 32 and validate on 20% of training data. **validation_split** monitors model performance on unseen data each epoch.



```
history = model.fit(x_train, y_train, epochs=10, batch_size=32, validation_split=0.2)
```

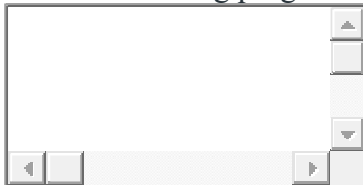
Output:

```
Epoch 1/10
1500/1500 ————— 7s 4ms/step - accuracy: 0.8721 - loss: 0.4363 - val_accuracy:
Epoch 2/10
1500/1500 ————— 9s 4ms/step - accuracy: 0.9624 - loss: 0.1212 - val_accuracy:
Epoch 3/10
1500/1500 ————— 6s 4ms/step - accuracy: 0.9737 - loss: 0.0864 - val_accuracy:
Epoch 4/10
1500/1500 ————— 6s 4ms/step - accuracy: 0.9799 - loss: 0.0664 - val_accuracy:
Epoch 5/10
1500/1500 ————— 6s 4ms/step - accuracy: 0.9838 - loss: 0.0526 - val_accuracy:
Epoch 6/10
1500/1500 ————— 10s 4ms/step - accuracy: 0.9880 - loss: 0.0397 - val_accuracy:
Epoch 7/10
1500/1500 ————— 7s 5ms/step - accuracy: 0.9907 - loss: 0.0338 - val_accuracy:
Epoch 8/10
1500/1500 ————— 10s 5ms/step - accuracy: 0.9920 - loss: 0.0284 - val_accuracy:
Epoch 9/10
1500/1500 ————— 6s 4ms/step - accuracy: 0.9928 - loss: 0.0242 - val_accuracy:
Epoch 10/10
1500/1500 ————— 11s 4ms/step - accuracy: 0.9933 - loss: 0.0214 - val_accuracy:
```

Training the Model

6. Evaluating and Visualizing Results

We evaluate test accuracy on unseen test data and plot training and validation loss curves to visualize learning progress.



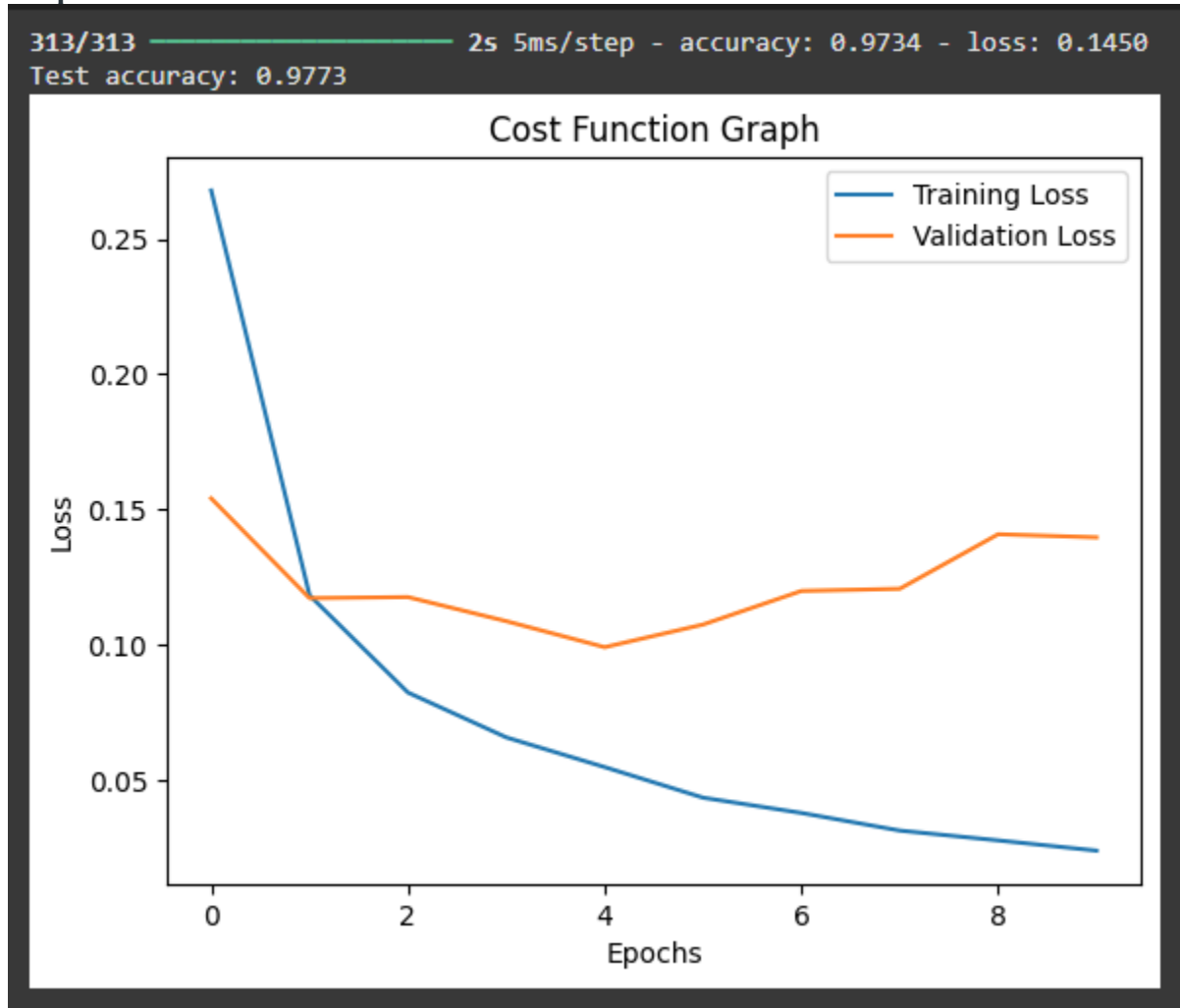
```
loss, accuracy = model.evaluate(x_test, y_test)
```

```
print(f'Test accuracy: {accuracy:.4f}')
```

```
plt.plot(history.history['loss'], label='Training Loss')
```

```
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Cost Function Graph')
plt.legend()
plt.show()
```

Output:



Evaluating and Visualizing Results

Advantages of RMSProp

- **Adaptive Learning Rates:** Adjusts learning rates for each parameter individually, optimizing updates more effectively.
- **Handles Non-Stationary Objectives:** Efficiently adapts to changing optimal parameter values over time.
- **Prevents Learning Rate Decay Problem:** Maintains optimal learning rates by using a decay rate unlike AdaGrad.
- **Improved Convergence Speed:** Faster convergence due to balanced and dynamic learning rates.

Disadvantages of RMSProp

- **Sensitivity to Hyperparameters:** Performance is sensitive to settings like decay rate and epsilon meaning it requires careful tuning.
- **Poor Performance with Sparse Data:** May struggle with sparse data, leading to slower or inconsistent convergence.

Momentum

Momentum is an optimization technique used to speed up the training of deep learning models by helping the model converge faster and avoid local minima. In traditional gradient descent, the weight updates depend solely on the current gradient. However, with momentum, the updates also consider the past gradients. This leads to smoother updates, especially in cases where the loss surface has steep slopes in some directions and flat regions in others. It simulates the idea of a ball rolling down a hill, where the movement gains speed and direction over time.

Example: In PyTorch, using momentum with SGD can be done as follows:

```
python
CopyEdit
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

Regularization Methods

Regularization techniques are used to reduce overfitting in deep learning models. Overfitting occurs when a model performs well on training data but poorly on unseen data. Two common regularization methods are L1 and L2 regularization. L1 regularization (also called Lasso) adds the sum of the absolute values of weights to the loss function, encouraging sparsity in the model. L2 regularization (also called Ridge) adds the sum of the squared values of weights, discouraging large weights but not necessarily forcing them to zero. Both techniques penalize complexity and promote simpler models.

Example: In Keras, L2 regularization can be applied as follows:

```
python
CopyEdit
from tensorflow.keras import regularizers
model.add(Dense(64, kernel_regularizer=regularizers.l2(0.01)))
```

Dropout

Dropout is a regularization technique that reduces overfitting by randomly "dropping" or disabling a fraction of neurons in the network during training. This prevents neurons from co-adapting too much to specific features in the training data. Dropout encourages the network to develop redundant representations so that the overall performance is more robust. During testing, all neurons are used, but the outputs are scaled to account for the training dropout.

Example: To apply dropout in Keras, use the following code:

```
python
CopyEdit
from tensorflow.keras.layers import Dropout
model.add(Dropout(0.5))
```

Batch Normalization

Batch Normalization (BatchNorm) is used to normalize the inputs of each layer so that they have zero mean and unit variance. This improves the training stability and speed. It also reduces the internal covariate shift, where the distribution of layer inputs changes during training. BatchNorm allows higher learning rates and can have a slight regularizing effect, sometimes reducing the need for dropout.

Example: In PyTorch, batch normalization is added like this:

```
python
CopyEdit
import torch.nn as nn
model = nn.Sequential(
    nn.Linear(128, 64),
    nn.BatchNorm1d(64),
    nn.ReLU()
)
```

Weight Decay

Weight decay is a regularization technique where a small penalty is added to the weights during optimization to prevent them from growing too large. It is mathematically equivalent to L2 regularization but is often implemented directly in the optimizer. This helps in keeping the model simpler and prevents overfitting by discouraging complex solutions with high-magnitude weights.

Example: In PyTorch, weight decay can be set as a parameter in the optimizer:

```
python
CopyEdit
optimizer = torch.optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-5)
```

Hyperparameter Tuning

Hyperparameter tuning is the process of finding the best combination of hyperparameters for a machine learning model. Hyperparameters include learning rate, batch size, number of layers, dropout rate, and more. Unlike model parameters, which are learned during training,

hyperparameters must be manually set before training. Tuning involves methods like grid search, random search, and Bayesian optimization to test different values and choose the one that gives the best validation performance.

Example: Using random search with Keras Tuner allows automatic exploration of hyperparameters:

```
python
CopyEdit
import keras_tuner as kt
tuner = kt.RandomSearch(build_model, objective='val_accuracy', max_trials=5)
tuner.search(x_train, y_train, epochs=10, validation_data=(x_val, y_val))
```